PROGRAM:

Import the required libraries

```
import numpy as np
import pandas as pd
import sklearn
import scipy
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report,accuracy_score
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from pylab import rcParams
rcParams['figure.figsize'] = 14, 8
RANDOM_SEED = 42
LABELS = ["Normal", "Fraud"]
```

Load the heart dataset

```
from google.colab import files
upload = files.upload()
```

Explore the dataset

```
data = pd.read_csv('creditcard.csv',sep=',')
data.head()
```

OUTPUT

	Time	V1	V2	V3	V7	V8	V9	...	V21	…	V28	Amt	Class	
0	0	-1.359807	-0.072781	0.462388	0.239599	0.098698	0.363787	...	…	0.133558	-0.021053	149.62	0
1	0	1.191857	0.266151	-0.082361	-0.078803	0.085102	-0.255425	...	…	-0.008983	0.014724	2.69	0
…	…	…	…	…	…	…	…	…	…	…				
4	2	-1.158233	0.877737	0.095921	0.592941	-0.270533	0.817739	...	…	0.219422	0.215153	69.99	0

5 rows × 31 columns

```
data.info()
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
```

```
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
             …
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

```
data.isnull().values.any()
```

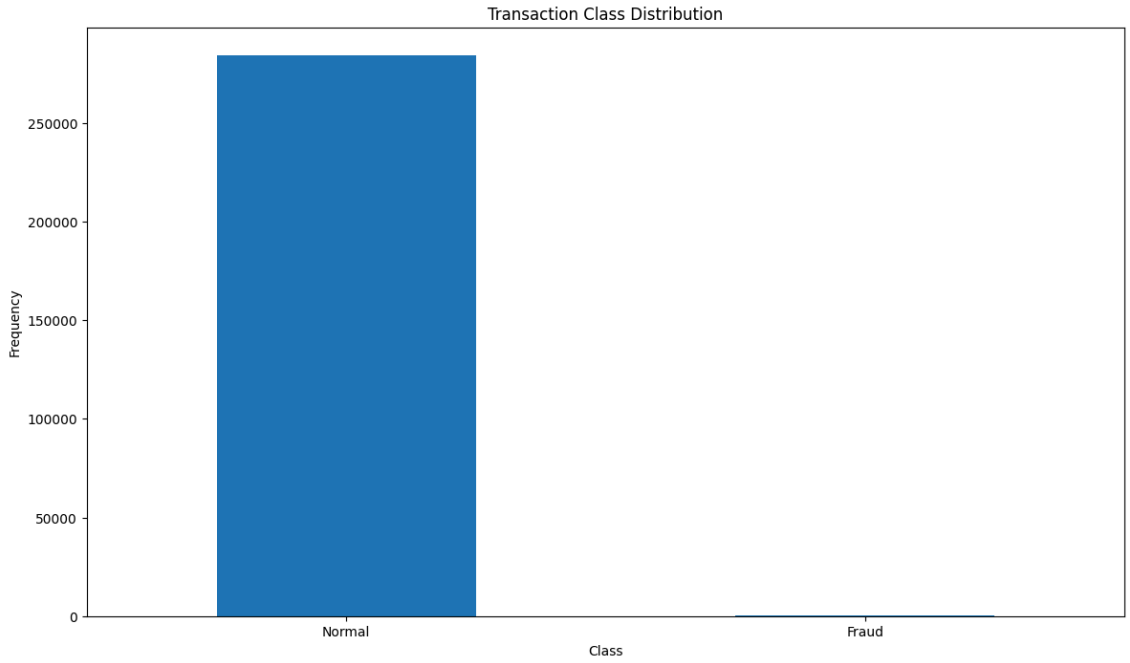
OUTPUT

```
np.False_
```

```
count_classes = pd.value_counts(data['Class'], sort = True)
count_classes.plot(kind = 'bar', rot=0)
plt.title("Transaction Class Distribution")
plt.xticks(range(2), LABELS)
plt.xlabel("Class")
plt.ylabel("Frequency")
```

OUTPUT

```
Text(0, 0.5, 'Frequency')
```



```
fraud = data[data['Class']==1]
normal = data[data['Class']==0]
print(fraud.shape,normal.shape)
```

OUTPUT

```
(492, 31) (284315, 31)
```

```
fraud.Amount.describe()
```

OUTPUT

	Amount
count	492
mean	122.211321
std	256.683288
min	0
25%	1
50%	9.25
75%	105.89
max	2125.87

dtype: float64

```
normal.Amount.describe()
```

OUTPUT

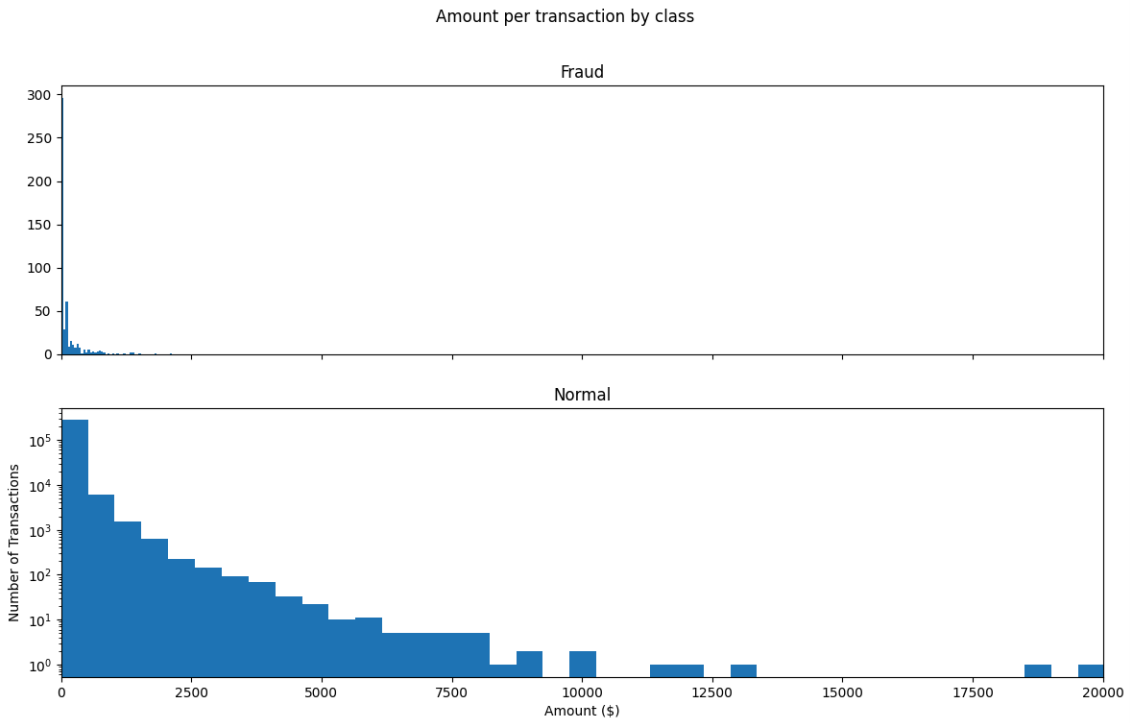
	Amount
count	284315
mean	88.291022
std	250.105092
min	0
25%	5.65
50%	22
75%	77.05
max	25691.16

dtype: float64

```
f, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
f.suptitle('Amount per transaction by class')
```

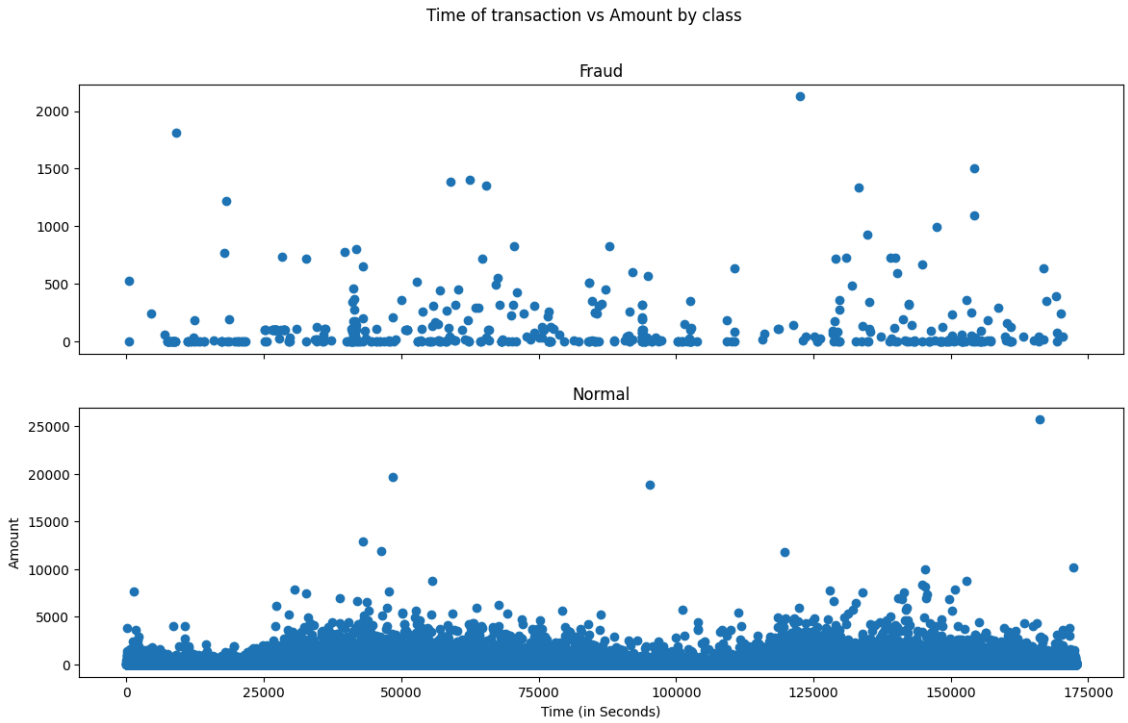
```
bins = 50
ax1.hist(fraud.Amount,bins = bins)
ax1.set_title('Fraud')
ax2.hist(normal.Amount,bins = bins)
ax2.set_title('Normal')
plt.xlabel('Amount ($)')
plt.ylabel('Number of Transactions')
plt.xlim((0,20000))
plt.yscale('log')
plt.show();
```

OUTPUT




```
f, (ax1, ax2) = plt.subplots(2, 1, sharex=True)
f.suptitle('Time of transaction vs Amount by class')
ax1.scatter(fraud.Time, fraud.Amount)
ax1.set_title('Fraud')
ax2.scatter(normal.Time, normal.Amount)
ax2.set_title('Normal')
plt.xlabel('Time (in Seconds)')
plt.ylabel('Amount')
plt.show()
```

OUTPUT




```
data= data.sample(frac = 0.1,random_state=1)
data.shape
```

OUTPUT

(28481, 31)

```
Fraud = data[data['Class']==1]
Valid = data[data['Class']==0]
outlier_fraction = len(Fraud)/float(len(Valid))
print(outlier_fraction)
print("Fraud Cases : {}".format(len(Fraud)))
print("Valid Cases : {}".format(len(Valid)))
```

OUTPUT

0.0017234102419808666

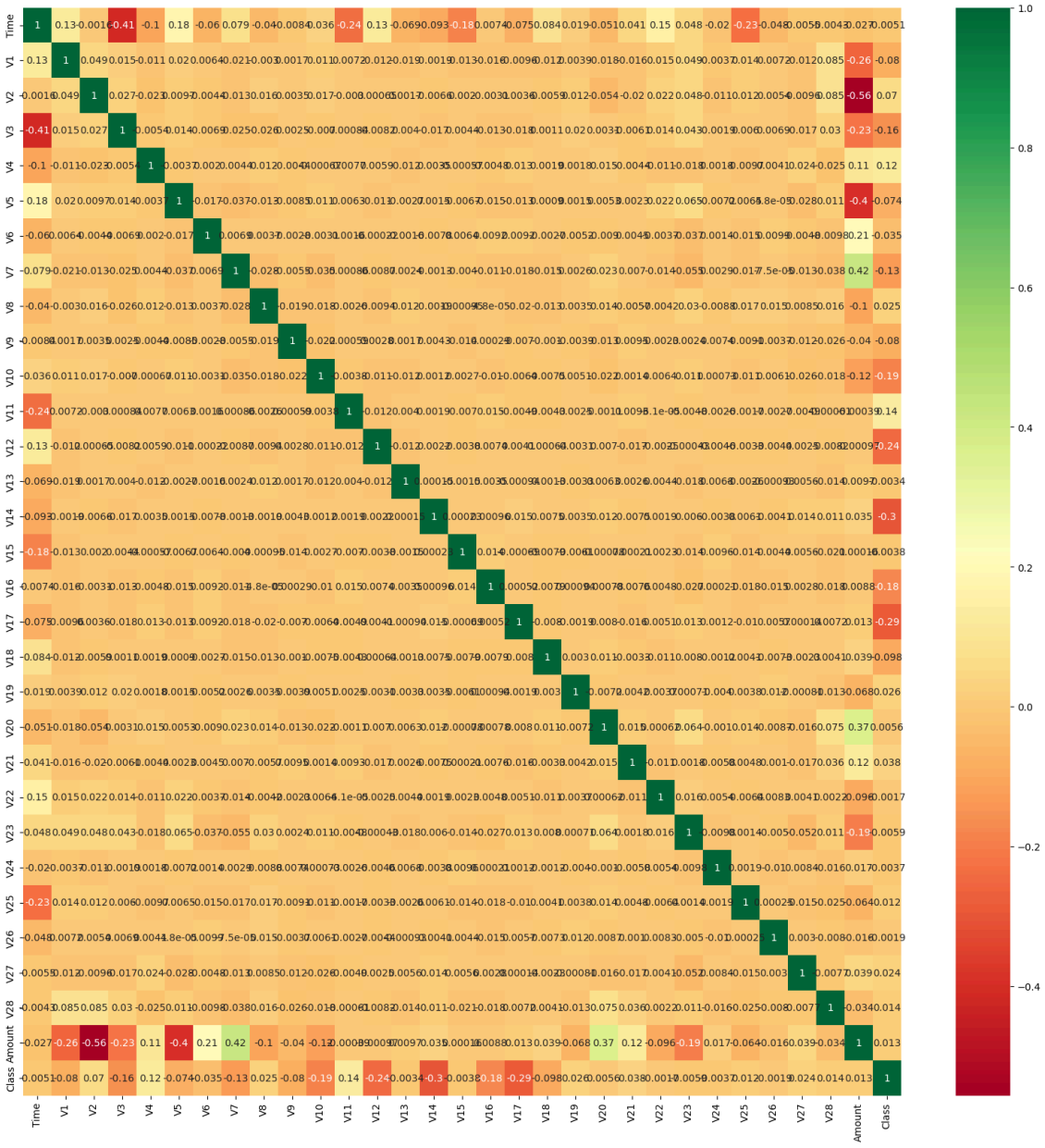
Fraud Cases : 49

Valid Cases : 28432

Perform Isolation Forest Algorithm and Local Outlier Factor(LOF) Algorithm to dataset.

```
import seaborn as sns
```

```
#get correlations of each features in dataset
corrmat = data.corr()
top_corr_features = corrmat.index
plt.figure(figsize=(20,20))
#plot heat map
g=sns.heatmap(data[top_corr_features].corr(),annot=True,cmap="RdYlGn")
```




```
columns = data.columns.tolist()
# Filter the columns to remove data we do not want
columns = [c for c in columns if c not in ["Class"]] # Store the variable we are predicting
target = "Class"
# Define a random state
```

```
state = np.random.RandomState(42)
X = data[columns]
Y = data[target]
X_outliers = state.uniform(low=0, high=1,
size=(X.shape[0],X.shape[1]))
# Print the shapes of X & Y
print(X.shape)
print(Y.shape)
```

OUTPUT

```
(28481, 30)
(28481,)
```

```
classifiers = {
    "Isolation Forest": IsolationForest(
        n_estimators=100,
        max_samples=len(X),
        contamination=outlier_fraction,
        random_state=state,
        verbose=0
    ),
    "Local Outlier Factor": LocalOutlierFactor(
        n_neighbors=20,
        algorithm='auto',
        leaf_size=30,
        metric='minkowski',
        p=2,
        metric_params=None,
        contamination=outlier_fraction
    )
}

type(classifiers)

n_outliers = len(Fraud)

for i, (clf_name, clf) in enumerate(classifiers.items()):
    # Fit the data and tag outliers
    if clf_name == "Local Outlier Factor":
```

```
        y_pred = clf.fit_predict(X)
        scores_prediction = clf.negative_outlier_factor_
    else:
        clf.fit(X)
        scores_prediction = clf.decision_function(X)
        y_pred = clf.predict(X)
    # Reshape the prediction values to 0 for Valid transactions, 1 for Fraud transactions
    y_pred[y_pred == 1] = 0
    y_pred[y_pred == -1] = 1
    n_errors = (y_pred != Y).sum()
    # Run Classification Metrics
    print("{}: {}".format(clf_name, n_errors))
    print("Accuracy Score :")
    print(accuracy_score(Y, y_pred))
    print("Classification Report :")
    print(classification_report(Y, y_pred))
```

OUTPUT

Isolation Forest: 73

Accuracy Score :

0.9974368877497279

Classification Report :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	28432
1	0.26	0.27	0.26	49
accuracy			1.00	28481
macro avg	0.63	0.63	0.63	28481
weighted avg	1.00	1.00	1.00	28481

Local Outlier Factor: 97

Accuracy Score :

0.9965942207085425

Classification Report :

	precision	recall	f1-score	support
0	1.00	1.00	1.00	28432
1	0.02	0.02	0.02	49
accuracy			1.00	28481
macro avg	0.51	0.51	0.51	28481
weighted avg	1.00	1.00	1.00	28481